

MICROSOFT FOUNDRY · MODEL ROUTER

One Endpoint, Smarter Spend

A hands-on deep-dive into the Foundry Model Router

Deploy. Evaluate. Optimize. Operate — with zero routing code.

*"We're paying frontier prices for FAQ-grade questions.
Can we route per request — without building our own
classifier?"*

— every team running an LLM in production, eventually

What Model Router is

A trained router model

Analyzes each prompt and selects the optimal backing model in real time.

One deployment

You call one endpoint. The router fans out to 28+ models across multiple providers.

Config, not code

Tune routing mode and model subset in the portal. No `route_intent()` to maintain.

Built-in failover + caching

Transparent retries on throttling. Automatic prompt caching on supported models.

What you'll be able to do

1. **Explain** how Model Router analyzes prompts and selects backing models
2. **Deploy** a router endpoint with routing mode and model-subset configuration
3. **Design** a representative prompt set spanning the complexity spectrum
4. **Compare** a frontier baseline against Model Router on quality, cost, latency
5. **Build** a custom Policy-Adherence evaluator with the Adaptive Evals rubric
6. **Optimize** by tuning modes, subsets, and prompt caching — config only
7. **Operate** with failover, continuous eval, and Foundry portal visibility

The scenario: Zava Travel Concierge

WORKLOAD

**FAQ · policy · trip
planning · expense ·
edge cases**

TODAY'S COST

~\$14k/mo

TARGET AFTER THIS WORKSHOP

~\$4.5k/mo

One frontier model is handling every request — from "what's the per-diem?" to "compare 2 hops vs. 1 international flight". The mix is exactly what Model Router was built for.

The arc — four scorecard moves

		Quality	Cost/task	p50 latency
v1	Frontier only (gpt-5)	4.3	\$0.028	3.2s
v2	Router Balanced	4.2	\$0.011	2.1s
v3	Router Cost	3.9	\$0.006	1.4s
v4	Router + subset + cache	4.3	\$0.009	1.8s ← WIN

Numbers are illustrative — you'll measure your own. Every row is one config change, measured before kept.

STAGE 1

Select

A router endpoint, deployed in minutes.

Lab 0 — Setup

What we deploy

- Foundry project in **Sweden Central**
- **Model Router** — Global Standard, Balanced mode
- **Baseline frontier model** (e.g. `gpt-5`)
- Credentials in `.env`

Verify both endpoints

```
az ai project create \  
  --name "zava-model-router" \  
  --resource-group "rg-zava-travel" \  
  --location "swedencentral"  
  
# Then deploy model-router via portal or CLI
```

Lab 1 — Build the prompt set

A workload's **distribution** is what gives Model Router its leverage.

Category	Count	Difficulty
simple_faq	15	easy
policy_question	10	easy/medium
trip_planning	10	medium
receipt_expense	5	medium
edge_case	5	hard
multi_step_reasoning	5	hard

If every prompt is equally complex, routing saves nothing. Real workloads have a long tail of easy queries — that's where the wins live.

STAGE 1 · CONT.

Deploy & configure

Two knobs: routing mode, model subset.

Lab 2 — Routing modes are a tolerance knob

Quality mode: "Always use the best model. Cost is irrelevant."
→ effectively frontier for everything (with failover)

Balanced mode: "Stay within ~2% of best quality. Minimize cost."
→ most prompts to capable-but-cheaper models
→ DEFAULT — best for most workloads

Cost mode: "Stay within ~6% of best quality. Maximize savings."
→ aggressive routing to cheapest sufficient model
→ best for high-volume, low-stakes work

Inspect the response metadata

```
from openai import AzureOpenAI

client = AzureOpenAI(...)
resp = client.chat.completions.create(
    model="model-router",
    messages=[{"role": "user", "content": prompt}],
)

print(resp.model)          # ← which underlying model handled it
print(resp.usage)          # ← tokens for cost calculation
```

The router tells you, per request, which backing model it picked. That's the signal you'll graph in Lab 3.

STAGE 2

Evaluate

Does it actually hold up on **your** workload?

Lab 3 — Baseline vs. Router

Three commands, three datasets

```
# 1. Frontier baseline
python code/run_comparison.py \
  --mode baseline \
  --dataset sample-data/router-eval-prompts.jsonl

# 2. Router (Balanced)
python code/run_comparison.py \
  --mode router \
  --dataset sample-data/router-eval-prompts.jsonl

# 3. LLM-as-a-judge scoring
python code/run_comparison.py --mode judge
```

What you measure

Response text + latency

v2 — first scorecard move

		Quality	Cost/task	p50 latency	
v1	Frontier only (gpt-5)	4.3	\$0.028	3.2s	
v2	Router Balanced	4.2	\$0.011	2.1s	✓

Quality dipped 0.1 of a point. Cost dropped 60%. Latency dropped 34%. The router is doing what it claimed — and we have the receipts.

The model distribution chart

"Which backing models did the router actually pick — and how often?"

This is the most revealing artifact of the entire workshop. For Zava Travel on Balanced mode you'll typically see:

- ~40% of prompts to a small/fast model (`gpt-5-mini` class)
- ~35% to a mid-tier model
- ~25% escalating to the frontier

*The distribution ***is*** your workload decomposition. You didn't have to design it — the router revealed it.*

Lab 4 — When generic quality isn't enough

*Quality 4.2 looks fine. But does the response cite the **right** reimbursement limit?*

Generic LLM-as-a-judge scores (Accuracy, Completeness, Clarity, Helpfulness) answer **"is this a good response?"**

They do **not** answer **"does this correctly apply our travel policy?"**

A smaller model can be clear, complete, and helpful — and still confabulate a \$400 per-diem that doesn't exist.

The Policy-Adherence rubric

Criterion	What it catches
Rule accuracy	Did it cite the correct section (hotels vs. flights)?
Completeness	Are all conditions and exceptions mentioned?
Boundary precision	Are dollar thresholds exact? (\$300 vs. \$325)
Hallucination absence	Did it invent rules not in the policy?

YAML rubric · weighted average · runs through Foundry Adaptive Evals.

Adaptive rules — context-aware weighting

```
adaptive_rules:
  # Budget questions → boundary precision matters most
  - condition: "input.category in ['budget_cap', 'reimbursement']"
    adjust_weights:
      boundary_precision: 0.40    # up from 0.25

  # Exception/approval questions → completeness matters most
  - condition: "input.category in ['exception', 'approval_chain']"
    adjust_weights:
      completeness: 0.40

  # Edge cases → hallucination risk is highest
  - condition: "input.category == 'edge_case'"
    adjust_weights:
      hallucination_absence: 0.40
```

STAGE 3

Optimize

Modes. Subset. Caching. Config only —
no rewrites.

Lab 5 — Compare all three modes

```
python code/run_mode_comparison.py --mode cost      --output results/router-cost/  
python code/run_mode_comparison.py --mode balanced --output results/router-balanced/  
python code/run_mode_comparison.py --mode quality  --output results/router-quality/
```

		Quality	Cost/task	p50 latency	
v2	Router Balanced	4.2	\$0.011	2.1s	
v3	Router Cost	3.9	\$0.006	1.4s	← too far
v4	Router Quality	4.4	\$0.025	3.0s	← like baseline

Cost mode saves more but drops a quality point. Quality mode regresses to baseline economics. Balanced is the sweet spot — for now.

Model subset — narrow the eligible set

```
Default subset: 28+ models across providers
Curated subset: gpt-5-mini · gpt-5 · claude-3.5-sonnet · llama-3-70b
                  ↑ chosen because:
                    - 2+ models for failover
                    - tool-calling support across the set
                    - per-token cost ceiling
                    - cache compatibility for the system prompt
```

A smaller subset = more predictable distribution = higher cache hit rate = compound wins.

Lab 6 — Prompt caching, automatically

Request 1: `system_prompt (2,000 tok) + user_A (50 tok)`
→ Router picks `gpt-5-mini`
→ Full processing at standard rate

Request 2: `system_prompt (2,000 tok) + user_B (60 tok)`
→ Router picks `gpt-5-mini` (same model!)
→ 2,000 cached tokens at reduced rate
→ ~50% savings on the system prompt portion

Request 3: `system_prompt (2,000 tok) + user_C (40 tok)`
→ Router picks `gpt-5` (different model)
→ No cache hit — caches are per-model

v4 — compound optimization wins

		Quality	Cost/task	p50 latency	
v1	Frontier only (gpt-5)	4.3	\$0.028	3.2s	
v2	Router Balanced	4.2	\$0.011	2.1s	
v3	Router Cost	3.9	\$0.006	1.4s	x rolled back
v4	Router + subset + cache	4.3	\$0.009	1.8s	✓ WIN

Same quality as the frontier baseline. ~68% less cost. ~44% less latency. Zero application code changed.

STAGE 4

Operate

Failover. Continuous eval. Portal visibility.

Lab 7 — Failover is built in

Normal flow:

Prompt → Router → selects gpt-5-mini → response ✓

Failover flow (gpt-5-mini throttled):

Prompt → Router → selects gpt-5-mini → 429 rate limit
→ transparently retries
→ selects gpt-5 → response ✓

Operational rule: your model subset **is** your fallback pool — never ship a subset of size 1.

Continuous evaluation in production

The loop

1. Sample N% of production traffic
2. Score with the Policy-Adherence evaluator
3. Track model distribution week over week
4. Alert on quality regression or distribution drift

Why it matters

- Router version updates add models — your mix will shift
- Traffic patterns drift as the product grows
- Catch regressions before customers do

RECAP

Three takeaways

1 — The router IS the decomposition

You don't have to guess which model fits which task.

Deploy Model Router, send representative prompts, **inspect the model distribution**. The router's choices empirically reveal your workload's structure.

The distribution chart is more valuable than the cost savings on the first run — it tells you what your workload actually looks like.

2 — Evaluation is the whole game

Without evaluation, routing is a guess.

With evaluation, routing is a measured optimization.

Generic quality

LLM-as-a-judge pairwise + absolute

Domain quality

Adaptive Policy-Adherence rubric

Cost

Router markup + per-model token pricing

Latency

Wall-clock p50 by category

3 — Optimization is config, not code

Every improvement in this workshop was a configuration change:

- Routing mode: **Balanced → Cost → Quality**
- Model subset: **all models → curated set**
- Prompt caching: **system-prompt design + subset for hits**

No application code was rewritten. The loop is **measure → tune config → measure again**.

When to use Model Router

Use it when

- Workload has diverse prompt complexity
- You want zero routing code to maintain
- Cost optimization is a real priority
- New models should help you automatically

Consider manual routing when

- You need deterministic model-per-task
- Strict per-task latency SLOs
- Compliance needs a model audit trail
- Workload is homogeneous (no wins)

Now you

Run it on your workload.

`workshops/model-router-demystified/` · runs in ~90 min

"Use the `run-workshop` skill on `workshops/model-router-demystified`."

One endpoint. Smarter spend. No routing code.